

PYTHON 90 DAYS CHALLENGE

DAY 07 • CONTROL FLOW

Nested Conditions | Ternary Operator | match-case | Truthy & Falsy

7.4 Nested Conditions

Definition: A nested condition means writing one condition inside another. The inner condition is checked only when the outer condition is **True**. It is useful when one decision depends on another decision.

Syntax

```
if outer_condition:
    if inner_condition:
        # statements
    else:
        # statements
else:
    # statements
```

Example

```
age = 20
has_ticket = True

if age >= 18:
    if has_ticket:
        print("Entry allowed")
    else:
        print("Ticket required")
else:
    print("Entry denied")
```

OUTPUT Entry allowed

How it works: Python first checks the age. Since age is 20, it then checks the ticket. Both conditions are satisfied, so entry is allowed.

7.5 Ternary Operator

The ternary operator is a short, one-line way to write a simple **if-else** statement. Use it when choosing between two values. For complex logic, normal if-else is easier to read.

Syntax

```
value_if_true if condition else value_if_false
```

Normal if-else

```
age = 20
if age >= 18:
    status = "Adult"
else:
    status = "Minor"
print(status)
```

OUTPUT Adult

Same code using ternary operator

```
age = 20
status = "Adult" if age >= 18 else "Minor"
print(status)
```

OUTPUT Adult

7.6 match-case

Definition: match-case compares one value with multiple possible cases. The case `_` pattern works like a default case and runs when no other case matches. It was introduced in Python 3.10.

Syntax

```
match value:
    case pattern1:
        # statements
    case pattern2:
        # statements
    case _:
        # default statements
```

Example

```
day = 3
match day:
    case 1:
        print("Monday")
    case 2:
        print("Tuesday")
    case 3:
        print("Wednesday")
    case _:
        print("Invalid day")
```

OUTPUT Wednesday

7.7 Truthy and Falsy Values

Python can evaluate a value as True or False inside a condition, even without a direct comparison such as `age >= 18`. A value that behaves like True is **truthy**; a value that behaves like False is **falsy**.

Common falsy values

VALUE	MEANING
False	Boolean false
0	Zero integer
0.0	Zero floating-point number
""	Empty string
None	No value / absence of a value

Common truthy values

VALUE	MEANING
True	Boolean true
10, -5	Non-zero numbers
"Python"	Non-empty string
" "	String containing a space

IMPORTANT: " " is truthy because it contains a space. It is not an empty string.

Example 1 — Non-empty string

```
name = "Rahul"
if name:
    print("Name is available")
```

```
else:  
    print("Name is missing")
```

OUTPUT Name is available

Example 2 — Empty string

```
name = ""  
if name:  
    print("Name is available")  
else:  
    print("Name is missing")
```

OUTPUT Name is missing

Example 3 — Number

```
amount = 0  
if amount:  
    print("Amount available")  
else:  
    print("Amount is zero")
```

OUTPUT Amount is zero